1. TRANSACTIONS IN DBMS

1. WHAT IS A TRANSACTION ?

- A transaction is a set of one or more database operations treated as a single logical unit of work.
- The operations can be read, insert, update, delete and must be executed together.
- It changes the database from one consistent state to another.
- If any operation fails, the entire transaction is rolled back (no partial changes).
- Follows ACID properties to ensure data integrity, consistency, concurrency and fault tolerance.

2. COMMON TRANSACTION OPERATIONS

Operation	Purpose
READ	Retrieve data from database (may use memory buffer/cache).
INSERT	Add new rows into a table.
UPDATE	Modify existing rows.
DELETE	Remove rows from a table.
COMMIT	Permanently save all changes (end transaction).
ROLLBACK	Undo all uncommitted changes (end transaction).

3. EXAMPLE - MONEY TRANSFER

Account A ₹5,000 —Transfer ₹1,000→ Account B ₹2,000

After successful transaction :

Account A	Account B
₹4,000 (-₹1,000)	₹3,000 (+₹1,000)

Both debit and credit must happen together as a single transaction.

4. TRANSACTION STATES

- Active (operations executing)
- Partially Committed (final operation executed)
- Committed (Commit successful)
- Failed / Aborted (Rollback / Recovery)

Failure can occur in active state also.
After commit or abort, transaction terminates.

5. TRANSACTION SCHEDULES

- **Serial Schedule** : Transactions run one after another (no interleaving).
- **Non-Serial Schedule** : Operations of multiple transactions may interleave. Concurrency control is required to maintain isolation and consistency.

6. KEY POINTS

- Transactions ensure data integrity, consistency and a valid state.
- Either all operations are committed or none (Atomicity).
- Committed changes are permanent (Durability).
- Handle concurrent execution (Isolation).
- If a transaction fails, rollback is performed.
- Transaction boundaries are important (if one commits, other failing later will not automatically roll it back).

7. ACID PROPERTIES (QUICK RECALL)

Property	Definition	Key Points	Example
Atomicity (A)	Either all operations commit or none do.	• All or nothing. • If failure occurs, rollback.	If debit succeeds but credit fails, both are rolled back.
Consistency (C)	Moves DB from one valid state to another, preserving rules and constraints.	• Maintains data integrity. • Enforces constraints. • No invalid state after transaction.	Bank balance cannot be negative.
Isolation (I)	Concurrent transactions are controlled so they do not interfere with each other.	• Supports safe concurrency. • Different isolation levels. • Prevents anomalies (dirty read, non-repeatable read, phantom read).	Two users updating same record should not cause incorrect results.
Durability (D)	Once committed, changes are permanent even if there is system crash or failure.	• Committed data is saved to disk (Write-Ahead Logging, etc.). • Survives crashes and recovery.	After commit, data remains even if database/server crashes.

8. COMMON CONCURRENCY ANOMALIES

Anomaly	What Happens ?	Example
Dirty Read	Read uncommitted data from another transaction.	T1 updates salary to 60k (not committed), T2 reads 60k.
Non-repeatable Read	Same query gives different results on repeated read.	T1 reads 50k, T2 updates to 60k.
Phantom Read	New rows appear in a repeated query.	T1 counts 10 orders, T2 inserts new order, T1 counts 11.
Lost Update	Concurrent updates overwrite each other.	T1 and T2 read same balance, one update is lost.

2. NORMALIZATION IN DBMS

1. WHAT IS NORMALIZATION ?

- Normalization is the process of organizing relational database tables based on dependencies to reduce redundancy and prevent data anomalies.
- It helps maintain data integrity and consistency by splitting data into logically related tables.
- It uses concepts like functional dependencies and normal forms (1NF, 2NF, 3NF, BCNF, 4NF, 5NF).

2. WHY DO WE NEED NORMALIZATION ?

- Avoids data redundancy.
- Prevents data anomalies (insert, update, delete).
- Improves data integrity and consistency.
- Easier to maintain and update.
- Better data storage and management.
- Forms a good base for scalable database design.

3. DATA ANOMALIES (Example)

order_id	product_id	product_name	customer_name
0101	P01	Laptop	Rahul
0102	P01	Laptop	Priya
0103	P01	Laptop	Aman

Problems :

- **Update anomaly** : Product name changed in only 2 rows.
- **Insert anomaly** : Cannot add new product without placing an order.
- **Delete anomaly** : Deleting last order removes product information.

4. FUNCTIONAL DEPENDENCY (FD)

- A → B means A uniquely determines B.
- For a given value of A, there is only one value of B.
- Examples :
  - product_id → product_name
  - customer_id → customer_name
  - order_id → customer_id

Types of Dependencies :

- **Full dependency** : depends on entire key.
- **Partial dependency** : depends on part of a composite key.
- **Transitive dependency** : A → B and B → C then A → C.

5. NORMAL FORMS WITH EXAMPLES

Normal Form	Key Rule / Definition	Example / Illustration
1NF (First NF)	• Each cell contains atomic (single) value. • No repeating groups or arrays. • All columns have single data type.	Before (Not in 1NF): order_id | products: 0101 | Laptop, Mouse After (1NF): order_id | product_id | product_name: 0101 | P01 | Laptop; 0101 | P02 | Mouse
2NF (Second NF)	• Must be in 1NF. • No partial dependency. • Non-key attributes should depend on the whole candidate key.	For key (order_id, product_id): product_name depends only on product_id (partial dependency). customer_id depends only on order_id (partial dependency).
3NF (Third NF)	• Must be in 2NF. • No transitive dependency. • Non-key attributes should depend only on the key ("key, the whole key and nothing but the key").	order_id → customer_id → customer_name customer_name is transitively dependent on order_id. So, keep customer_name in a separate table.
BCNF (Boyce-Codd NF)	• Every determinant of a non-trivial functional dependency must be a superkey.	If instructor_id → course_id and instructor_id is not a superkey, then table violates BCNF. (Separate instructor-course table).
4NF (Fourth NF)	• Every non-trivial multivalued dependency must have a superkey as determinant.	Employee has multiple skills and multiple languages (independent): Employee_Skills(emp_id, skill) Employee_Languages(emp_id, language)
5NF (Fifth NF)	• Every non-trivial join dependency must follow from candidate keys.	Used in complex relationships where table can be losslessly decomposed into multiple tables without redundancy.

6. NORMALIZATION EXAMPLE (STEP BY STEP)

Initial Table (Not Normalized):

order_id, product_id, product_name, customer_id, customer_name, quantity

Key = (order_id, product_id)

After 2NF & 3NF decomposition:

orders	order_items	products	customers
order_id (PK) customer_id (FK)	order_id (FK) product_id (FK) quantity	product_id (PK) product_name	customer_id (PK) customer_name

7. NORMALIZATION vs DENORMALIZATION

Normalization (OLTP)

- Reduces redundancy.
- Prevents anomalies.
- More tables (normalized).
- Easier to maintain.
- Strong data integrity.
- Common in OLTP systems.

Denormalization (OLAP / DW)

- Introduces controlled redundancy.
- Fewer joins, simpler queries.
- Better query performance.
- Common in data warehouses (dimensions, facts).
- Used for analytical workloads.
- Consistency managed through ETL/pipelines.

8. KEY TAKEAWAYS

- ✓ Transactions follow ACID properties.
- ✓ Understand functional dependencies and normal forms.
- ✓ Choose between normalized and denormalized design based on use case (OLTP vs OLAP).
- ✓ Ensure lossless decomposition and dependency preservation.
- ✓ In Data Engineering, often normalized in source (OLTP) and denormalized in warehouse for analytics.
- ✓ Data Engineering reduces redundancy and anomalies.

2. INTERVIEW SCENARIOS — TRANSACTIONS & NORMALIZATION

Detailed expected answers + follow-up traps

HOW TO ANSWER SCENARIO QUESTIONS

1. **Identify grain / business rule.**
   - Understand what each row represents.
   - Ask clarifying questions if needed.
2. **Find dependency or transaction boundary.**
   - Identify functional / multivalued dependencies.
   - For transactions, identify what is in the same transaction boundary.
3. **Propose table split / control logic.**
   - Show final tables, keys and relationships.
   - Mention transaction control or data flow if needed.
4. **Mention validation + trade-off.**
   - Explain how to verify correctness.
   - Discuss trade-offs (performance, complexity, etc.).

1. NORMALIZE AN ORDERS TABLE

Scenario

Table has columns order_id, product_id, product_name, customer_id, customer_name, quantity. Composite key is (order_id, product_id).

What interviewer expects

- Identify partial and transitive dependencies.
- Show proper table decomposition.

Answer

- product_name depends only on product_id → partial dependency (on part of composite key).
- customer_name depends on customer_id, if order_id determines customer_id, then customer_name is transitively dependent on order_id.
- Split into: orders(order_id, customer_id), order_items(order_id, product_id, quantity), products(product_id, product_name), customers(customer_id, customer_name).
- Validate joins reconstruct original business view without duplication.

Follow-up trap

- If product appears multiple times in same order, define proper grain or use order_line_id instead of (order_id, product_id).

Original Table (not normalized)

order_id	product_id	product_name	customer_id	customer_name	quantity
0101	P01	Laptop	C001	Rahul	2
0101	P02	Mouse	C001	Rahul	1
0102	P01	Laptop	C002	Priya	1

↓ Decomposition

orders

order_id	customer_id
0101	C001
0102	C002

order_items

order_id	product_id	quantity
0101	P01	2
0101	P02	1
0102	P01	1

products

product_id	product_name
P01	Laptop
P02	Mouse

customers

customer_id	customer_name
C001	Rahul
C002	Priya

2. WHY NOT ALWAYS NORMALIZE FOR ANALYTICS ?

Scenario

E-commerce system has slow reporting across many normalized OLTP tables.

What interviewer expects

- Design trade-off, not blind normalization.
- Understand performance and workload differences.

Answer

- Keep OLTP normalized for write integrity and to avoid update anomalies.
- For analytics, consider denormalized reporting layer / data warehouse / star schema / aggregates.
- First check indexes, query plan, and workload separation (reports should not hit OLTP).
- Denormalize selectively for performance and simpler BI queries.

Follow-up trap

Denormalization can reintroduce redundancy and requires reliable refresh logic (ETL/ELT). Stale or incorrect data is a risk.

Normalized OLTP vs Denormalized OLAP (Analytics)

Normalized OLTP (Transactional)	Denormalized OLAP (Analytical)
• Multiple related tables (3NF).	• Fewer, wider tables (star schema).
• Minimal redundancy.	• Some redundancy (e.g., include names).
• Optimized for writes (insert/update/delete).	• Optimized for read and aggregation.
• Maintains data integrity.	• Simpler queries for BI tools.
• Complex joins for reporting.	• Precomputed aggregates / summary tables.
• Slower for large analytical queries.	• Needs ETL/refresh logic to keep data current.

3. DOES THIS VIOLATE 3NF ?

Scenario

Customer table has customer_id, postal_code, city.

What interviewer expects

- Reason using functional dependency.
- Decide if 3NF is violated and how to fix it.

Answer

- If business rule is customer_id → postal_code and postal_code → city, then city is transitively dependent on customer_id.
- This violates 3NF (non-key attribute depending on another non-key attribute).
- Separate postal reference table if this dependency truly holds.
- If one postal_code maps to multiple cities, then postal_code → city does not hold and decomposition changes (need to keep city with postal_code or a different model).

Follow-up trap

Never normalize unless the dependency is actually valid in the business data. Always verify with real data and business rules.

Functional Dependency Example

Business rule (example):

1. customer_id → postal_code
2. postal_code → city

↓

Therefore:
customer_id → city
(transitive dependency)

Decomposition (if FD holds)

customers

customer_id	postal_code
C001	110001
C002	560001

postal_codes

postal_code	city
110001	Delhi
560001	Bangalore

4. ORDERS COMMIT, INVENTORY FAILS — WHAT HAPPENS ?

Scenario

Pipeline writes to orders and inventory. Orders commits successfully, inventory update fails afterward.

What interviewer expects

- Transaction boundary reasoning.
- Understand ACID scope and handling strategies.

Answer

- ACID rollback applies only within the same transaction boundary.
- If both writes are in one transaction and commit has not happened, both will roll back on failure.
- If orders committed independently and inventory fails later, ACID will NOT automatically undo orders.
- In distributed workflows, use retries, idempotency, compensating actions, or supported distributed transaction patterns (e.g., two-phase commit, saga).

Follow-up trap

ACID does not solve cross-system workflow correctness by itself. If different systems/databases are involved, additional patterns (idempotency, compensation) are required.

Two Possible Flows

A. Same Transaction (ACID)

Start Transaction → Write Orders → Update Inventory → Commit

- If inventory update fails before commit, entire transaction is rolled back.
- No partial data (atomicity).

B. Separate Transactions

Write Orders (Commit) → Update Inventory (fails) → Inventory remains unchanged but Orders is committed

- ACID does not automatically undo orders.
- Use retries, compensating actions, or manual recovery.

5. EMPLOYEE SKILLS × LANGUAGES COMBINATION

Scenario

Table stores employee_id, skill, language and every combination appears.

What interviewer expects

- Recognize multivalued dependency / 4NF-style issue.
- Show proper decomposition and reasoning.

Answer

- If skills and languages are independent, storing all combinations creates redundancy (multivalued dependency).
- Split into employee_skills(employee_id, skill) and employee_languages(employee_id, language).
- This removes artificial combinations and update anomalies.
- If a skill is valid only for a specific language, then the relationship is not independent and decomposition must respect that business rule.

Follow-up trap

Always verify whether attributes are independent before decomposing. If there is a real dependency between skill and language, keep them together.

Original Table (Every combination stored)

employee_id	skill	language
E01	Java	English
E01	Java	Hindi
E01	Python	English
E01	Python	Hindi

Problem

- Skills and languages are independent.
- Every combination creates redundancy.
- Leads to insert, delete and update anomalies.

Decomposition (if independent)

employee_skills

employee_id	skill
E01	Java
E01	Python

employee_languages

employee_id	language
E01	English
E01	Hindi

FINAL INTERVIEWER CHECKLIST

1. **State the dependency clearly.**
   - Identify functional or multivalued dependency, or transaction boundary.
   - Explain the business rule.
2. **Mention anomalies or transaction boundary.**
   - For normalization: talk about partial, transitive or multivalued dependency.
   - For transactions: identify what is within the same transaction and what is not.
3. **Show the final tables / control logic.**
   - Provide table structure, keys and relationships.
   - For transactional scenarios, show the flow and handling mechanism.
4. **Explain validation.**
   - How to verify joins reconstruct original data without duplication.
   - For transactions, how to test failure cases (rollback, recovery, etc.).
5. **Mention trade-offs, not just textbook rules.**
   - Discuss performance vs. normalization, complexity, business requirements.
   - Acknowledge alternative approaches where applicable.